# 1 Evaluation Methodology

This chapter details the methodology used to evaluate the **Agentic Project Management (APM)** framework. To assess the framework's effectiveness against current industry standards, we conducted a comparative case study involving human participants.

The evaluation was designed to measure two distinct dimensions of efficiency:

- **Economic Efficiency** (monetary and computational cost)
- **Process Efficiency** (human cognitive load and interaction burden)

To accurately capture these distinct metrics, the study was divided into two complementary sub-studies, leveraging the unique data-collection capabilities of two different IDE platforms (Cursor and VSCode with GitHub Copilot).

## 1.1 Research Objectives

The primary objective of this research is to determine whether the structural overhead introduced by APM (i.e., the Setup and Task Loop phases) yields a tangible return on investment compared to single or double-agent workflows.

We define the following Research Questions (RQs):

- **RQ1 (Economic Efficiency):** Does the multi-agent structure of APM significantly increase the token consumption and financial cost compared to single-agent or double-agent workflows?
- **RQ2 (Process Efficiency):**
  - Does the structured coordination of APM reduce the "human burden" during execution? Specifically, does it reduce the need for user interventions, corrections, and clarifications?
  - Is the reduction in human burden due to the structured coordination of APM significant enough to justify the structural overhead?
  - Are the agents able to effectively coordinate and complete the task without the need for constant human oversight?
- **RQ3 (Functional Efficacy):**
  - Can APM successfully deliver complex, non-trivial features in scenarios where unstructured agentic workflows typically encounter "Agentic Handoff Failures"?
  - How does the performance of APM compare to the performance of single-agent and double-agent workflows?

## 1.2 Testbed and Task Design

To evaluate the frameworks under realistic constraints, we utilized an existing open-source project, the `eclass-mcp-server`, a Python-based Model Context Protocol (MCP) server designed to interface with the `openeclass` platform. Openeclass is an online platform for Greek universities that allows students to manage their courses and assignments.

### 1.2.1 The Task:

The final task requires participants to implement a new `get_course_announcements` tool in the Python MCP server and, crucially, to create a complete native C replica of the MCP server. The task specification was refined through iteration to ensure sufficient complexity for differentiating multi-agent from single-agent approaches.[1]

This C implementation adds significant complexity, turning the assignment into a challenge of managing code translation, memory safety, and protocol details across two very different languages. As such, the task is specifically designed to stress-test the limits of agent coordination and reveal differences between multi-agent and single-agent approaches, especially in scenarios where context management becomes a bottleneck.

**Key Technical Challenges:**

1. **Dynamic Data Source:** The required data for implementing the new feature is not available in static HTML for parsing. Agents must research the legacy codebase to discover a specific AJAX endpoint and its behavior.
2. **Nested Parsing:** The server returns JSON where the payload is an array of raw HTML strings, requiring a second layer of parsing to extract the data.
3. **Client-Side Filtering:** The legacy API does not support date filtering. The agent must implement robust client-side logic to parse and filter unique locale-formatted dates.
4. **C Implementation:** Agents must build a native C MCP server replica, manually implementing JSON-RPC 2.0 with standard C string manipulation (no external JSON libraries such as cJSON/Jansson). Use of `libcurl` for HTTP/HTTPS and `libxml2` for HTML parsing, was required for the eclass MCP server functionality.
5. **Memory Safety:** The C implementation must be memory-safe, passing Valgrind checks without leaks or errors.

**Deliverables:**

1. Core server logic in modular Python files (existing and new functionality).
2. A native C implementation of the MCP server with equivalent (existing and new) functionality.
3. A working build system (Makefile) for the C replica.

[1] Task specification and grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task

### 1.2.2 Grading:

Submissions are evaluated using an automated black-box test suite with comprehensive validation, grading, metric extraction, and dataset aggregation capabilities (see the evaluation appendix for detailed infrastructure documentation).[2]

The test suite awards **48 points total**: 12 points for baseline functionality (existing Python tools), 12 points for the new Python feature (`get_course_announcements`), and 24 points for the complete C replica implementation. A submission with no patch applied receives the baseline score of **12/48 points**.

All tests are run against fixed baseline commits to ensure reproducibility: `eclass-mcp-server` at commit `8b4e39c` and `openeclass` at commit `e8b3329` (Docker image `openeclass:test`).

### 1.2.3 Repository Architecture and Data Management

The case study employs a two-repository system separating public resources from sensitive participant data. The **task repository** (`bsc-apm-case-study-task`) is openly accessible and includes task specifications, grading infrastructure, and participant guides. The **dataset repository** (`bsc-apm-case-study-data`) contains all submitted participant data: code patches, sanitized chat transcripts (see Section 1.2.4), and anonymized session metadata.

Access to the dataset repository's secret access key requires explicit authorization and compliance with data usage agreements. This controlled access mechanism enables peer review and reproducibility while maintaining participant privacy protections.

### 1.2.4 Sanitization and PII Handling

Chat exports contain sensitive information (unstructured user-input text, file paths, command histories, timestamps). To protect participant privacy while enabling reproducibility, all exports are sanitized using reversible encryption before storage.

Each chat is assigned a unique identifier via SHA256 hashing of concatenated user messages for anonymous cataloging. Essential metadata (chat ID, model IDs, request count) is preserved in plaintext to enable dataset organization without decryption, while conversation content is encrypted using an HMAC-SHA256 XOR cipher. The same sanitization method is applied uniformly across both platforms.

For detailed technical specifications of the anonymization and encryption processes, see the evaluation appendix (Section **??**).

[2] Grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task/tree/main/grader

## 1.3 The Two Sub-Studies:

To provide a holistic evaluation, we split the experiment into two sub-studies. This design allowed us to isolate specific variables and leverage the distinct data collection and logging capabilities of each tool.

Table 1: Comparison of Sub-Study Designs

| **Feature** | **Sub-Study A (Cursor)** | **Sub-Study B (GitHub Copilot)** |
|---|---|---|
| **Primary Focus** | Quantitative Economic Analysis | Qualitative Process Analysis |
| **Target Audience** | Controlled, homogeneous group with Cursor Premium features | Broader cohort with Copilot Premium features, primarily University students |
| **Primary Metric** | Total tokens (input/output), total cost ($) | Interaction patterns, user/system action burden |
| **Data Source** | Exported usage statistics (CSV/JSON) from the Cursor dashboard, filtered by session | Exported GitHub Copilot JSON chat logs capturing message details, tool calls, timestamps |

### 1.3.1 Sub-Study A: Quantitative Economic Analysis (Cursor)

This study focused on **RQ1**. It utilized a smaller, controlled group of participants with access to Cursor's premium AI features.

- **Rationale:** Cursor participants export CSV usage statistics from the Cursor dashboard and filter them by session start time using a provided Python script to isolate session-specific metrics (see Section **??** for detailed workflow).
- **Model Standardization:** To ensure rigorous comparison, all participants were strictly required to use **Cursor's Composer 1** model for the entire duration of the study. This eliminates model variance as a confounding variable for cost.

### 1.3.2 Sub-Study B: Qualitative Process Analysis (GitHub Copilot)

This study focused on **RQ2**. It utilized a larger cohort of mainly University students and professionals with access to GitHub Copilot's premium AI features (via the GitHub Education Pack or regular premium subscription).

- **Rationale:** GitHub Copilot provides detailed JSON chat exports that include metadata on tool calls, timestamps, and speaker roles. This granular data allows for programmatic analysis of interaction patterns that is not possible with Cursor's limited chat export functionality.
- **Hybrid Model Strategy:** To mimic real-world usage constraints (request quotas), participants in the APM group followed a hybrid strategy: using a Premium Model (Claude Sonnet 4) for high-level planning and a Budget Model (with a 0x request multiplier) for low-level execution.

## 1.4 Frameworks Under Evaluation

Three distinct workflow configurations were evaluated across both sub-studies:

1. **Agentic Project Management (APM):** The proposed framework. It enforces a strict separation of concerns:
    - *Setup Agent:* Project discovery and planning.
    - *Manager Agent:* Coordination, task assignment, and review.
    - *Implementation Agent:* Execution and memory logging.
2. **GitHub Spec-kit:** A structured, "constitutional" baseline. It utilizes a single-agent workflow guided by rigid templates and three sequential commands (`specify`, `plan`, `tasks`).
3. **Plan Mode/Plan Agent (Baseline):** The industry-standard "First plan, then execute" feature native to the IDE. Cursor provides **Plan Mode**, while GitHub Copilot provides the **Plan Agent**. Both represent the current state-of-the-art for AI-powered planning workflows.

## 1.5 Metrics and Data Collection

### 1.5.1 Study A Metrics (Cursor)

Study A addresses **RQ1** (Economic Efficiency) and provides supplementary process efficiency data from chat exports.

**Economic Metrics** - Collected via Cursor's CSV usage statistics:

- **Total Input Tokens:** The volume of context processed by the models during the session.
- **Total Output Tokens:** The volume of code and text generated during the session.
- **Total Cached Tokens:** The volume of tokens served from Cursor's cache, reducing cost.
- **Total Cost ($):** The financial cost of the session based on Cursor's API pricing.
- **Time to Completion:** The total elapsed time from task initiation to final submission, measured in minutes.

**Process Efficiency Metrics** - Derived from parsing Cursor markdown chat exports:

- **Total Requests:** Sum of all user messages across all chat files.
- **Behavior Distribution:** Ratios of Short/Protocol/Standard inputs (same classification scheme as Study B, using 50-character threshold and APM protocol signatures).
- **Behavior Magnitude:** Average characters per input category (Short/Protocol/Standard).

### 1.5.2 Study B Metrics (GitHub Copilot)

Derived from parsing sanitized GitHub Copilot JSON chat transcripts to address **RQ2**. APM's multi-agent workflow spans multiple chat sessions (Setup, Manager, and Implementation agents), while other evaluated frameworks use single or double-agent workflows. To ensure fair comparison, metrics are computed by aggregating all session files belonging to a single participant submission into a unified project-level view, capturing the total interaction cost regardless of how the work was distributed across sessions.

Metrics are organized into four tiers:
**Session Metadata:**

- **Unique Model IDs:** The distinct language models used across all sessions. Used to verify adherence to the Hybrid Model Strategy (e.g., `copilot/claude-sonnet-4` for planning, `copilot/gpt-4.1` for execution).
- **Global Start/End:** The earliest and latest timestamps across all sessions, used to compute project-level duration.

**Reliability Metrics (Tier 1)** - Measures of System Competence:

- **Tool Success Rate:** Ratio of successfully completed tool invocations to total invocations. Measures agent autonomy and ability to execute without failure.
- **Tool Retry Rate:** Ratio of tool invocation retries to total tool execution rounds. Indicates frequency of self-correction loops during execution.
- **User Edit Rate:** Ratio of user-edited terminal commands to total terminal invocations. A direct proxy for correction burden; higher values indicate the agent frequently generated incorrect commands that required manual intervention by the user.

**Volume Metrics (Tier 2)** - Measures of Project Cost:

- **Total Requests:** Sum of all user messages across all session files. Also serves as a cost proxy, since GitHub Copilot charges per request/credit in premium subscription tiers.
- **Session Duration:** Global wall-clock time (Project End Time - Project Start Time), measured in minutes. Captures the real-world time cost of the entire workflow. Calculated as the difference between the earliest and latest timestamps across all sessions.

**Behavior Distribution (Tier 3)** - Measures of Process Efficiency:

- **Ratio of Short Inputs:** Fraction of user inputs with fewer than 50 characters. Represents "supervisor" behavior; quick approvals/ confirmations/ acknowledgements that require minimal cognitive effort.
- **Ratio of Protocol Inputs:** Fraction of user inputs containing APM-specific signatures (e.g., YAML frontmatter keys, meta-prompt headers). Represents the administrative overhead of the framework: mechanical copy-paste operations required for agent coordination. This metric represents the "routing" responsibilities that are inherent in the APM framework.

- **Ratio of Standard Inputs:** Fraction of user inputs that are natural language without APM signatures. Represents the intervention burden; actual debugging, clarifications, and corrections. This metric isolates genuine user effort from the mechanical overhead captured by Protocol inputs.

**Behavior Magnitude (Tier 4)** - Depth of Interaction:

- **Average Characters per Bucket:** Mean character count for Short, Protocol, and Standard input categories. Provides context for the distribution ratios. For example, confirming that Protocol inputs are large (full task assignments) while Short inputs are minimal (single-word approvals).

### 1.5.3 Functional Metrics (Both Studies)

To address **RQ3**, final code artifacts (`.patch` files) were evaluated against the automated test suite described in the Grading subsection (48 points total). The suite validates both Python and C implementations across three dimensions:

- **Baseline Regression:** Ensures existing functionality (12 points) remains intact after patch application, critical for measuring framework impact on code stability.
- **Functional Correctness:** End-to-end validation of new feature implementation (12 points) and complete C replica (24 points) against specification requirements.
- **Robustness:** Edge case handling including authentication failures, invalid inputs, empty responses, and memory safety (Valgrind validation for C implementation).

# 2 Results

## 2.1 Functional Performance Analysis

**Primary Metrics:** Total score distribution, completion rates (48/48, 24-47, 12-23, baseline-only), baseline regression rates (python_baseline_score $< 12$), feature implementation success (Python feature and C replica completion rates).
**Category Analysis:** Success rates per test category (build, protocol, login, logout, courses, authstatus, announcements) for both Python and C implementations. Identifies framework-specific failure patterns and strengths.
**Comparative Analysis:** T-tests comparing APM vs. Spec-kit and APM vs. Plan Agent for total score, category scores, and completion rates. Effect sizes (Cohen's d) to assess practical significance.

## 2.2 Economic and Process Efficiency Analysis

**Study A (Economic):** Mean cost, token consumption, and efficiency metrics (cost per point, tokens per point) per framework. Comparative analysis of APM vs. baselines with correlation to functional scores.
**Study B (Process):** Reliability metrics (tool_success_rate, user_edit_rate), behavior distribution (ratio_short, ratio_protocol, ratio_standard), and volume metrics (total_requests, session_duration). Correlation analysis between process metrics and functional scores.

## 2.3 Learning Effects and Confounding

**Critical Issue:** Participants complete the same task twice (Session 1 and Session 2) using different frameworks. This creates a learning effect where task familiarity from S1 influences S2 performance, potentially confounding framework comparisons.
**Analysis Strategy:**

- **Learning Effect Matrix:** Calculate mean improvement (percentage change) from S1 to S2 for all S1→S2 framework combinations across all metrics (functional scores, cost, process efficiency)
- **S1 Framework Effect:** Compare mean improvement when S1 is APM vs. Spec-kit vs. Plan Agent (averaged across all S2 frameworks). Tests whether the S1 framework affects task learning quality.
- **S2 Framework Effect:** Compare mean improvement when S2 is APM vs. Spec-kit vs. Plan Agent (averaged across all S1 frameworks). Tests whether the S2 framework affects how much improvement is visible.
- **Order Effect:** For each framework pair, compare both directions (e.g., APM→Spec-kit vs. Spec-kit→APM). Tests whether order of framework exposure affects learning.
- **Baseline Learning:** Overall mean improvement across all combinations (pure task learning effect regardless of framework)

**Key Questions:** Does task learning confound framework comparisons? Which S1→S2 combinations show strongest learning effects? Is there a "best first framework" that maximizes S2 improvement? How do we control for learning effects when comparing frameworks?

# 3 Discussion

## 3.1 Cost-Benefit Analysis: Context Overhead vs. Failure Prevention

Synthesize economic metrics (Study A) with functional success rates to assess whether APM's higher cost is justified by reduced failure rates. Key analyses: (1) Cost-benefit: Compare mean cost of failed baseline submissions vs. APM's mean cost (including successes). (2) ROI: Calculate cost per point for each framework. (3) Failure cost: Quantify "wasted cost" of baseline failures vs. APM's cost premium. (4) Consistency: Compare variance in cost and scores across frameworks.
**Key Questions:** Is APM's cost increase justified by reduced failure rates? What is the break-even point? Does APM prevent costly "Agentic Handoff Failures"? Is the cost increase linear or exponential?

## 3.2 Process Efficiency: Protocol Overhead vs. Intervention Reduction

Synthesize process metrics (Study B) to assess whether APM's protocol overhead (ratio_protocol) is offset by reduced intervention burden (ratio_standard, user_edit_rate). Key analyses: (1) Net cognitive load: Compare ratio_protocol + ratio_standard across frameworks. (2) Role shift: Compare ratio_short (supervisor behavior) across frameworks. (3) Autonomy: Correlate tool_success_rate with user_edit_rate. (4) Time efficiency: Compare session_duration and requests per minute.
**Key Questions:** Does APM's Manager agent reduce coordination burden? Is protocol overhead offset by reduced intervention? Do APM users shift from "debugger" to "supervisor" role? Is total cognitive load lower for APM?

## 3.3 Functional Efficacy: Reliability and Failure Patterns

Synthesize functional scores, completion rates, and category breakdowns to assess APM's reliability. Key analyses: (1) Success rates: Percentage achieving full completion (48/48) per framework. (2) Baseline regression: Compare regression rates (python_baseline_score $< 12$). (3) Failure patterns: Identify which test categories fail most per framework. (4) Consistency: Calculate coefficient of variation (SD/mean) for scores.
**Key Questions:** Does APM achieve higher functional success rates? Does APM prevent catastrophic failures? Is APM more consistent (lower variance)? Does APM's structured approach help with specific challenges (C implementation, memory safety)?

## 3.4 Learning Effects: Confounding and Framework Transferability

Address how task learning (S1→S2 improvement) may confound framework comparisons. Key analyses: (1) Learning effect magnitude: Overall mean improvement across all S1→S2 combinations (baseline task learning). (2) S1 framework effect: Does the S1 framework affect task learning quality? (3) S2 framework effect: Does the S2 framework affect how much improvement

is visible? (4) Order effects: Compare framework pairs in both directions (e.g., APM→Spec-kit vs. Spec-kit→APM). (5) Framework transferability: Does learning APM in S1 improve S2 performance regardless of S2 framework?
**Key Questions:** How do we control for learning effects when comparing frameworks? Which S1→S2 combinations show strongest learning? Is there a "best first framework" for task learning? Do learning effects confound framework comparisons?

### 3.5 Hybrid Model Strategy and Platform Generalizability

Assess whether APM's hybrid model strategy (premium planning, budget execution) maintained quality while reducing cost. Verify strategy adherence via unique_model_ids and compare functional scores vs. single-model baselines. Discuss platform differences (Cursor vs. Copilot) and model effects (Composer 1 vs. Claude Sonnet 4) on generalizability.

### 3.6 Threats to Validity

- **Learning Effects:** Addressed via counterbalancing (Tool A/Tool B order).
- **Model Variance:** Mitigated in Study A by standardizing on Composer 1. Addressed in Study B by using a Claude Sonnet 4 for the planning phase of all tools, and a budget model for the implementation phase of all tools.
- **Participant Experience:** Variance in student skill levels (mitigated by the "Detailed PRD" which acts as a leveling mechanism and the "Default Prompt" containing hints and guidance for the agents).